SYSTEM DESIGN

Database Indexing Visual Guide

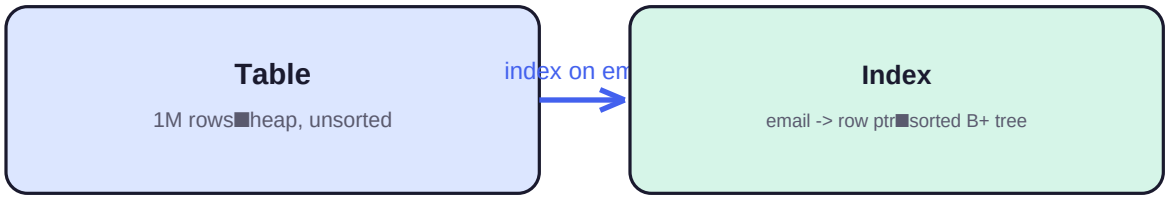
Find rows without scanning tables:
index types, composite keys, and the write tax.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Index Idea

A table of contents for your table: sorted pointers to rows.



- No index: full table scan -- read all 1M rows to find one email. With index: 3-4 page reads.
- The index is a SEPARATE structure the planner may use; the table (heap) stays untouched.
- Indexes speed reads and SLOW writes: every INSERT/UPDATE/DELETE maintains each index.

STEP 2/6

B-tree vs Hash vs GIN

Different structures for different questions.

B-tree (default)

equality + ranges + ORDER BY -- the all-rounder

Hash

equality only, no ordering -- niche, rarely better than B-tree

GIN

inverted index: full-text search, JSONB keys, arrays

GiST / BRIN

geo shapes, or huge naturally-ordered tables (time series)

Default to B-tree. Reach for GIN when you query INSIDE a document (JSONB) or text. Reach for BRIN for append-only time data.

STEP 3/6

Composite Indexes

One index on (last_name, first_name): order of columns is the whole game.

```
CREATE INDEX ON users (last_name, first_name);
```

- Leftmost prefix rule: the index serves WHERE last_name=? AND first_name=?, and WHERE last_name=? alone.
- It does NOT serve WHERE first_name=? alone -- the leading column must be constrained.
- Put the most selective / most-queried column first; equality columns before range columns.
- Example: (tenant_id, created_at) -- equality on tenant, range on time: perfect for SaaS queries.

STEP 4/6

Covering Indexes

INCLUDE the columns the query needs: answer without touching the table.

```
SELECT name FROM users WHERE email = ?;

CREATE INDEX ON users (email) INCLUDE (name);
```

No covering

index -> heap fetch■random I/O per row

Covering

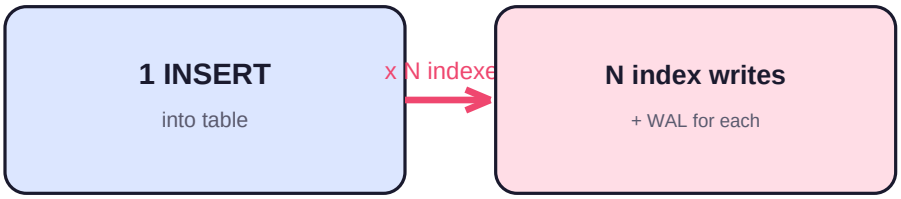
index-only scan■no heap visit

EXPLAIN shows 'Index Only Scan'. Watch visibility-map staleness: recently updated pages still force heap checks.

STEP 5/6

The Write Tax

Every index is a second (third, fourth) thing to maintain per write.



- 5 indexes = ~6x the write I/O of the bare row (plus lock contention on hot index pages).
- Unused indexes are pure tax: find them (pg_stat_user_indexes: zero scans) and drop them.
- Bulk loads: drop indexes, load, rebuild -- 10x faster than maintaining them row by row.
- Partial indexes (WHERE active) index only hot rows: smaller, faster, cheaper to maintain.

STEP 6/6

Reading EXPLAIN

The planner tells you which index it chose -- and its cost math.

```
Seq Scan on users  (cost=0.00..1834.00 rows=1)
  Filter: (email = 'a@b.com')   <- BAD: scanning

Index Scan using users_email_idx (cost=0.42..8.44)
  Index Cond: (email = 'a@b.com') <- GOOD: seeking
```

- Seq Scan on a big table with a selective filter = missing index (usually).
- Rows estimate vs actual (EXPLAIN ANALYZE): bad stats -> bad plans -> run ANALYZE.
- Index Cond vs Filter: Cond seeks the tree; Filter scans and discards -- Cond is what you want.

STEP ✓I✓

Key Takeaways

Database Indexing — Remember

- Indexes are separate sorted structures: 3-4 page reads instead of full table scans.
- B-tree is the default; GIN for documents/text/arrays; BRIN for append-only time data.
- Composite indexes obey the leftmost-prefix rule -- order columns by query pattern.
- Covering indexes (INCLUDE) answer queries without visiting the heap.
- Every index taxes writes: drop unused ones, use partial indexes, and verify with EXPLAIN.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev